OPEN LIVE APPLICATION | wsm-project-seven.vercel.app

WEB SEARCH AND MINING / PRACTICAL REPORT

Information Retrieval System

A modular search application with Boolean and wildcard retrieval

Prepared by

Dhavala Dinesh Kumar

Branch: CSE A | Student ID: b124043

20 documents | 50 lines per document | 40 passing tests

November 2026

Project Overview

This assignment builds a working Information Retrieval System that loads a local text corpus, preprocesses documents, creates an inverted index, and retrieves matching files through a Flask web interface. The design keeps each processing responsibility in a small module so the search pipeline is clear and practical to demonstrate.

The completed corpus contains 20 topic-focused documents. Each file has exactly 50 non-empty lines. The application builds its index at startup and supports single-term, Boolean, and wildcard searches against the real corpus.

Project at a glance

Area	Implementation
Backend	Python 3 and Flask serve the interface and JSON search endpoint.
Preprocessing	Alphanumeric tokenization, lowercase case folding, and consistent query normalization.
Index	Dictionary-based inverted index with document postings and term frequencies.
Retrieval	AND, OR, NOT, AND NOT, prefix, suffix, and infix wildcard queries.
Frontend	Responsive HTML, CSS, and JavaScript interface with read-only search results.
Verified corpus	20 text files, 50 non-empty lines each, and 40 passing automated tests.

Design goals

The system favors clear module boundaries, predictable search behavior, and output that explains why a document matched. The interface displays document IDs, filenames, snippets, result counts, and wildcard expansions without allowing result text to be edited.

Assignment objectives and outcomes

Objective	Evidence in this project
Build a document pipeline	Load, tokenize, normalize, and index 20 corpus files in a repeatable startup sequence.
Apply IR data structures	Use a vocabulary and inverted postings with per-document term frequencies.
Support retrieval models	Evaluate Boolean operators with set algebra and expand wildcard terms against the vocabulary.
Demonstrate end-to-end search	Connect a responsive browser interface to Flask routes and verify the corpus with automated tests.

Architecture and Modules

At startup, the application reads every .txt file in the documents folder, assigns IDs in natural filename order, and builds a single inverted index. A query then follows the same token and normalization rules before the processor evaluates its operators.

Module	Responsibility
Document loader	Reads text files, stores filenames and contents, assigns DOC1 through DOC20, and tolerates missing or empty files.
Tokenizer	Extracts alphanumeric word tokens and removes punctuation from ordinary text.
Normalizer	Trims whitespace and folds tokens to lowercase so query and corpus terms match consistently.
Inverted index	Maps each vocabulary term to document postings and records term frequency.
Query processor	Parses Boolean expressions and evaluates postings with set intersection, union, and difference.
Wildcard processor	Expands asterisk patterns against the indexed vocabulary and unions matching postings.
Flask and frontend	Connects the search page to the live /search API and renders actual backend results.

Search pipeline

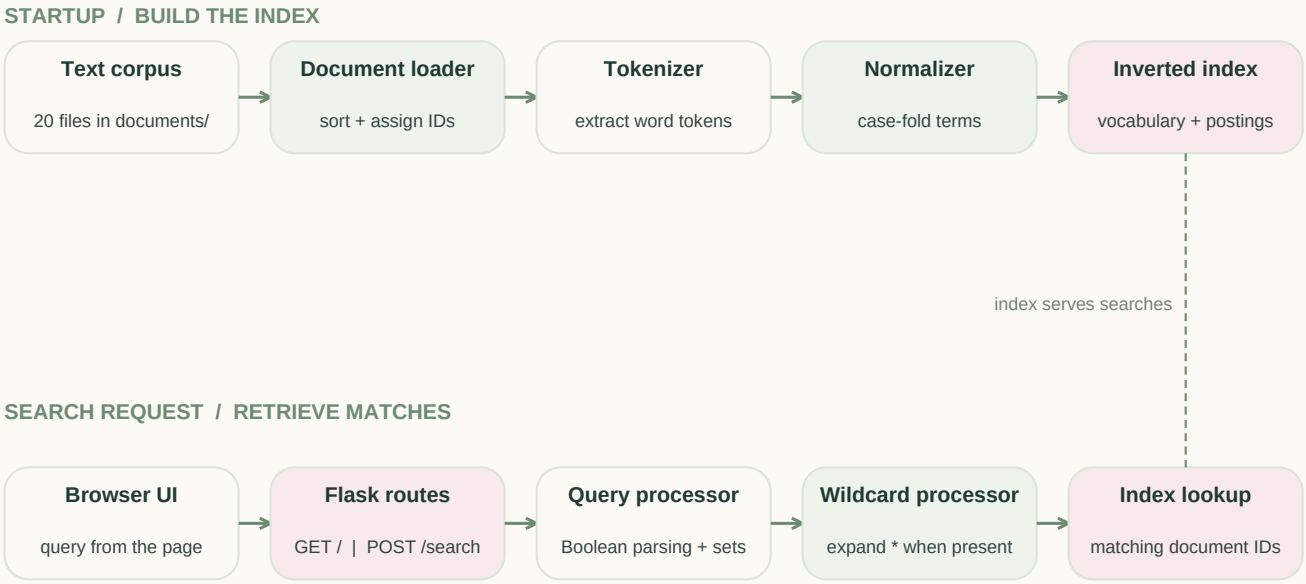
1. Load files from documents/ and assign stable document IDs.
2. Tokenize and normalize document contents, then build the vocabulary and postings.
3. Receive a query from the browser and apply the same text normalization.
4. Expand wildcard terms when present and evaluate Boolean set operations.
5. Return matching IDs, filenames, snippets, totals, and expanded terms to the page.

Why the modules are separated

Design choice	Benefit during search
Shared token rules	Documents and queries normalize consistently, avoiding case-related misses.
Postings as sets	AND, OR, and NOT map directly to intersection, union, and complement.
Plain-text corpus	New documents can be added without changing retrieval modules.

Architecture Diagram

The diagram separates startup indexing from request-time retrieval. On startup, the loader and text-processing modules build the shared index. During a search, Flask passes the request to the query processor, which resolves terms through postings and delegates wildcard expansion when needed.



The app returns the matching document IDs, filenames, snippets, wildcard expansions, and timing to the browser as JSON.

Working Directory and File Guide

This tree describes the assignment checkout used to build and run the project. The documents directory is the corpus; modules contains the IR pipeline; templates and static contain the browser interface; tests contains automated checks. The output and tmp folders hold generated report artifacts and intermediate render files.

```
D:/wsm assinement/
|-- app.py                      Flask entry point and HTTP routes
|-- config.py                   Paths and local server settings
|-- requirements.txt            Python package requirements
|-- vercel.json                 Vercel Python/static routing configuration
|-- .gitignore                  Generated files and local secrets exclusions
|-- api/
|   `-- index.py                Serverless entry point for Vercel
|-- modules/
|   |-- __init__.py             IR package
|   |-- document_loader.py      Read, sort, and identify documents
|   |-- tokenizer.py            Extract alphanumeric tokens
|   |-- normalizer.py           Normalize tokens and query terms
|   |-- inverted_index.py       Build vocabulary and postings
|   |-- query_processor.py      Parse and evaluate Boolean queries
|   `-- wildcard_processor.py   Expand * patterns against vocabulary
|-- documents/                  20 corpus files; 50 non-empty lines each
|   |-- doc1.txt
|   |-- doc2.txt
|   |-- doc3.txt
|   |-- doc4.txt
|   |-- doc5.txt
|   |-- doc6.txt
|   |-- doc7.txt
|   |-- doc8.txt
|   |-- doc9.txt
|   |-- doc10.txt
|   |-- doc11.txt
|   |-- doc12.txt
|   |-- doc13.txt
|   |-- doc14.txt
|   |-- doc15.txt
|   |-- doc16.txt
|   |-- doc17.txt
|   |-- doc18.txt
|   |-- doc19.txt
|   `-- doc20.txt
|-- templates/
|   `-- index.html              Search page markup
|-- static/
|   |-- style.css               Responsive visual design
|   `-- script.js               Browser search and result rendering
|-- tests/
|   `-- test_search.py          Automated module and retrieval tests
|-- README.md                   Setup and project usage
|-- REPORT.md                   Assignment report source
|-- output/pdf/
|   `-- WSM_Assignment_Report.pdf Generated report artifact
|-- tmp/pdfs/build_report.py    Reproducible PDF builder
|-- .vercel/                    Local Vercel project link metadata
`-- .env.local                  Local deployment environment settings
```

Local run and verification commands

```
cd "D:/wsm assinement"
python -m pip install -r requirements.txt
python app.py
python -m pytest tests/test_search.py -v

Local browser: http://127.0.0.1:5000
```

Implementation Details

Document preparation and index construction

DocumentLoader scans the configured corpus directory for .txt files and applies a natural numeric sort so doc2.txt is assigned before doc10.txt. It creates stable identifiers (DOC1, DOC2, DOC3, and onward) and metadata containing filenames, paths, text, character counts, and empty-state information. The index builder tokenizes each document, normalizes each term, and aggregates occurrence counts per document.

The resulting inverted index is a dictionary from normalized vocabulary terms to postings. Each posting maps a document ID to that term's frequency in the document. This gives the query processor a direct lookup for a term and a set of document IDs for Boolean operations.

Term	Posting example	Meaning
python	{DOC1: 4, DOC5: 2}	Python occurs four times in DOC1 and twice in DOC5.
search	{DOC2: 1, DOC3: 3}	Search occurs in both documents, with the shown frequencies.

Query parsing and Boolean evaluation

The query processor recognizes terms, parentheses, and the AND, OR, and NOT operators without case sensitivity. It inserts an implicit AND between adjacent terms where appropriate, converts infix expressions to Reverse Polish Notation using the Shunting-Yard method, then evaluates the expression with a stack of document-ID sets. AND is set intersection, OR is set union, and unary NOT subtracts a posting set from the universe of loaded document IDs. Parentheses allow explicit grouping.

```
Query:       python AND (search OR retrieval)
Tokens:      python AND ( search OR retrieval )
RPN:         python search retrieval OR AND
Evaluation:  postings(python) intersection
             (postings(search) union postings(retrieval))
```

Wildcard expansion

A wildcard term is normalized and split at each asterisk. Literal segments are escaped before being joined with a regular-expression match-any segment, then anchored so the full vocabulary term must match. The processor sorts matching vocabulary terms, unions their postings, and returns both the matched documents and the expansion list so the interface can explain the result.

Web Interface and API

The browser page is served from templates/index.html. static/style.css provides responsive styling and static/script.js submits searches with fetch, reads the JSON response, and builds result cards. The page reports status, number of matches, elapsed time, snippets, filenames, and wildcard expansions. Returned text is inserted as text content, rather than interpreted as HTML.

Route	Method	Purpose and response
/	**GET**	Serve the search page and provide the loaded document count to the template.
/search	**POST**	Accept JSON containing query; return matches, metadata, snippets, parsed tokens, wildcard expansions, status, and execution time.
/index-info	**GET**	Return corpus count, vocabulary size, and per-document metadata for diagnostics.

Example request and response shape

```
POST /search
Content-Type: application/json

{"query": "python AND search"}

{
  "query": "python AND search",
  "matching_doc_ids": ["DOC1"],
  "total_matching": 1,
  "expanded_wildcards": {},
  "documents": [{"doc_id": "DOC1", "filename": "doc1.txt",
                 "snippet": "...", "term_frequencies": {"python": 4}}],
  "status": "success",
  "execution_time_ms": 0.5
}
```

Deployment layout

For local development, app.py creates the Flask application and runs the development server using settings from config.py. For Vercel, api/index.py exposes the application as a serverless handler, while vercel.json routes static assets and application requests. The production URL recorded during verification is wsm-project-seven.vercel.app; Vercel access protection may require an authenticated session.

Configuration and boundaries

config.py derives corpus, template, and static paths from the project base directory, which keeps the checkout relocatable. requirements.txt lists the runtime dependency. The system keeps the corpus as plain text and builds its index in process memory at startup; it does not use an external database or third-party search engine.

Query Processing and Examples

The query processor supports explicit Boolean operators. AND has higher precedence than OR, while parentheses can group expressions. A leading NOT returns documents in the loaded collection that do not contain the following term. Wildcard expressions are matched against the current vocabulary before their document postings are combined.

Query	Behavior
python	Returns documents containing the normalized term python.
python AND search	Returns documents containing both terms.
python OR java	Returns documents containing either term.
NOT java	Returns documents that do not contain java.
python AND NOT java	Keeps python matches and removes java matches.
comp*	Expands to vocabulary terms beginning with comp, including computation.
***tion**	Matches vocabulary terms ending in tion, including information.
inter*net	Expands to internet and retrieves its matching document.

Results and error handling

A valid query with no matches produces a clear no-results state. Empty queries receive a warning instead of being evaluated as a real search. Wildcard results show the expanded vocabulary terms so users can inspect how the pattern was interpreted.

The browser sends JSON to Flask and renders returned values as text nodes. This keeps the interface connected to the actual index and avoids treating search text as markup.

Worked Boolean evaluation

For **python AND (search OR retrieval)**, the processor first groups the OR expression, then intersects those matches with the python postings. This makes the query's meaning explicit and demonstrates operator precedence.

Stage	Value
Normalized tokens	python AND (search OR retrieval)
Postfix / RPN	python search retrieval OR AND
Set expression	P intersect (S union R)
Result	Documents containing python and at least one of search or retrieval.

Wildcard trace: inter*net

The asterisk matches any character sequence between the literal prefix **inter** and suffix **net**. In the sample vocabulary it expands to **internet**; the system then unions the posting list for that term and reports the expansion to the page.

Sample Corpus

The corpus spans programming, search, data structures, language processing, machine learning, and interface design. Topic overlap is intentional: it makes Boolean operations meaningful and gives wildcard patterns a real vocabulary to expand against.

ID	Topic	Key concepts
DOC1	**Python programming**	Syntax, functions, modules, testing
DOC2	**Information retrieval**	Queries, postings, normalization
DOC3	**Search engines**	Crawling, ranking, internet pages
DOC4	**Data structures**	Arrays, trees, graphs, computation
DOC5	**Machine learning**	Features, training, evaluation
DOC6	**Web crawling**	URLs, fetching, responsible access
DOC7	**Search ranking**	TF-IDF, PageRank, relevance
DOC8	**Boolean retrieval**	AND, OR, NOT, set operations
DOC9	**Database indexes**	B-trees, hashes, full-text search
DOC10	**Text normalization**	Tokens, case folding, punctuation
DOC11	**Natural language processing**	Entities, extraction, classification
DOC12	**Learning approaches**	Labels, features, model evaluation
DOC13	**Semantic search**	Embeddings, vectors, hybrid retrieval
DOC14	**Data structure applications**	Maps, heaps, query postings
DOC15	**Graph algorithms**	Traversal, PageRank, recommendation
DOC16	**Retrieval evaluation**	Precision, recall, ranking metrics
DOC17	**Distributed search**	Shards, replicas, query coordination
DOC18	**Query expansion**	Synonyms, feedback, suggestions
DOC19	**Digital libraries**	Metadata, access, preservation
DOC20	**Accessible search**	Keyboard use, responsive results, labels

Every file contains exactly 50 non-empty lines. The loader assigns IDs from filenames in natural numeric order, so doc2.txt remains DOC2 and doc20.txt remains DOC20.

Testing and Verification

The automated suite covers the main backend modules and search operations. It also enforces the corpus size, natural filename ordering, and the 50-line requirement for every text file.

Test area	Coverage	Result
Document loading	20 files, IDs, filenames, non-empty content, natural order, 50 lines each	**PASS**
Text preparation	Tokenization, punctuation, case folding, whitespace	**PASS**
Inverted index	Vocabulary, postings, term frequencies, unique IDs	**PASS**
Boolean retrieval	Single term, AND, OR, NOT, AND NOT, empty and no-match queries	**PASS**
Wildcard retrieval	Prefix comp*, suffix *tion, infix inter*net, actual result retrieval	**PASS**
Full suite	40 automated tests	**40 / 40**

Live deployment

The Vercel production deployment is READY at wsm-project-seven.vercel.app. The production page displays 20 documents, /index-info returns DOC1 through DOC20, and the live inter*net request expands to internet and returns DOC3.

The local Flask page runs at http://127.0.0.1:5000. Vercel Authentication protection is enabled for the production project, so visitors may need to sign in.

Test distribution

Test class	Coverage	Count
TestDocumentLoader	Reading, IDs, order, empty or missing inputs	**8**
TestTokenizer	Word extraction, punctuation, edge inputs	**6**
TestNormalizer	Case folding and whitespace	**6**
TestInvertedIndex	Vocabulary, postings, term frequency	**6**
TestQueryProcessor	Boolean parsing and result sets	**8**
TestWildcardProcessor	Prefix, suffix, infix expansion	**6**

Verification checkpoints

The checks validate both component behavior and assignment-specific constraints: all 20 filenames load in numeric order, each document contains 50 non-empty lines, searches return IDs from the loaded corpus, and wildcard matches are resolved from the actual vocabulary. The deployment smoke check confirms the index metadata and a live wildcard request.

Installation and Use

The application runs with Python 3 and Flask. From the project directory, install its dependency and start the development server:

```
pip install -r requirements.txt
python app.py

Open http://127.0.0.1:5000
```

The index is rebuilt whenever the application starts. Add or edit .txt files in documents/ and restart the server to include the changes.

Key takeaways

The project demonstrates an end-to-end modular information retrieval pipeline.

The corpus contains 20 sample documents, each with exactly 50 non-empty lines.

Boolean operations use postings sets; wildcard patterns expand against the vocabulary.

The Flask frontend displays results generated from the actual document index.

All 40 automated tests pass, including corpus and wildcard regression checks.

Conclusion

This assignment brings document loading, preprocessing, indexing, Boolean query processing, wildcard expansion, and a responsive GUI into one working system. Its modular structure makes the processing steps visible and explainable, while the 20-document corpus provides a substantial set of examples for a WSM practical demonstration.

Scope and future improvements

Current scope	Possible extension
Boolean matching returns documents that satisfy the expression.	Add ranked retrieval such as TF-IDF to order results by relevance.
The index is built in memory when the app starts.	Persist the index or add incremental updates for larger corpora.
The corpus contains plain-text documents.	Add metadata, phrase search, or fielded queries.

Prepared by Dhavala Dinesh Kumar | CSE A | b124043